Project - Low Level Design

On

Marketing Content Generator

Course Name: **Generative AI**

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	YASH YADAV	EN22CS3011113
2	AYUSH MISHRA	EN22EL301020
3	VISHAL	EN22CS3011091
4	ASHUTOSH KUMAR	EN22EL301019
5	HARSH MANDLOI	EN22ME304035
6	AYUSH MODI	EN21CS301196

Group Name: 13D10

Project Number: GAI-26

Industry Mentor Name: Mr. Yash

University Mentor Name:Prop Divya Kumawat

Academic Year: 2025-2026

Table of Contents

1. Introduction

This Low Level Design (LLD) document provides a detailed internal view of the Marketing Content Generator, a GenAI-powered application. It describes the design of each module, class, method, and data structure to ensure developers can understand, maintain, or extend the system. The system focuses on modularity, statelessness per request, and extensibility.

1.1 Scope of the Document

The scope includes the detailed design of:

- LLMEngine: The core interface for Groq API interactions.
- ContentVectorStore: Management of the ChromaDB vector database.
- Prompt Templates: Structured engineering for various marketing content types.
- Streamlit UI: The user interface and session management.
- File Exporter: Utility functions for document serialization.

1.2 Intended Audience

This document is intended for developers, system architects, and maintainers who need to understand the internal logic, database schema, API contracts, and inter-module communication of the project.

1.3 System Overview

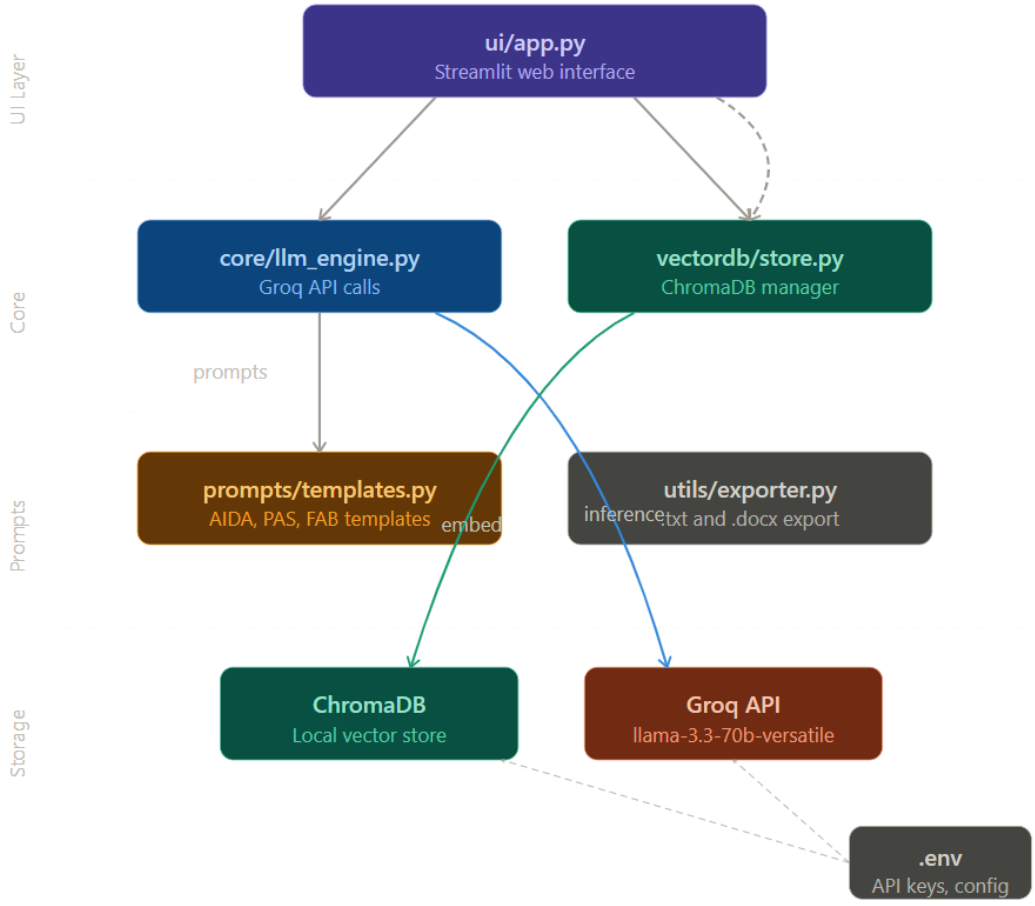
The Marketing Content Generator is a Python-based application using **Groq LLM** for text generation and **ChromaDB** for semantic memory. It utilizes a 5-layer unidirectional pipeline where data flows from user input through the prompt engine, LLM, and vector store, and finally back to the UI.

2. System Design

2.1 Application Design

The application is structured into five distinct layers:

1. L1 (UI): ui/app.py – Collects inputs and renders outputs.
2. L2 (Prompts): prompts/templates.py – Builds structured LLM prompts.
3. L3 (Engine): core/llm_engine.py – Handles Groq API communication.
4. L4 (Vector DB): vectordb/store.py – Manages embeddings and persistence.
5. L5 (Utils): utils/exporter.py – Formats content for .txt and .docx.



2.2 Process Flow

The standard generation flow follows these steps:

1. Input Validation: Ensures topic, audience, and USP fields are non-empty.
2. Vector Lookup: Searches ChromaDB for similar past content to ensure brand consistency.
3. LLM Call: Constructs a dual-message prompt (system + user) for the Groq API.
4. Persistence: Saves the newly generated content into the vector store.
5. Rendering: Updates Streamlit session state to display the result.

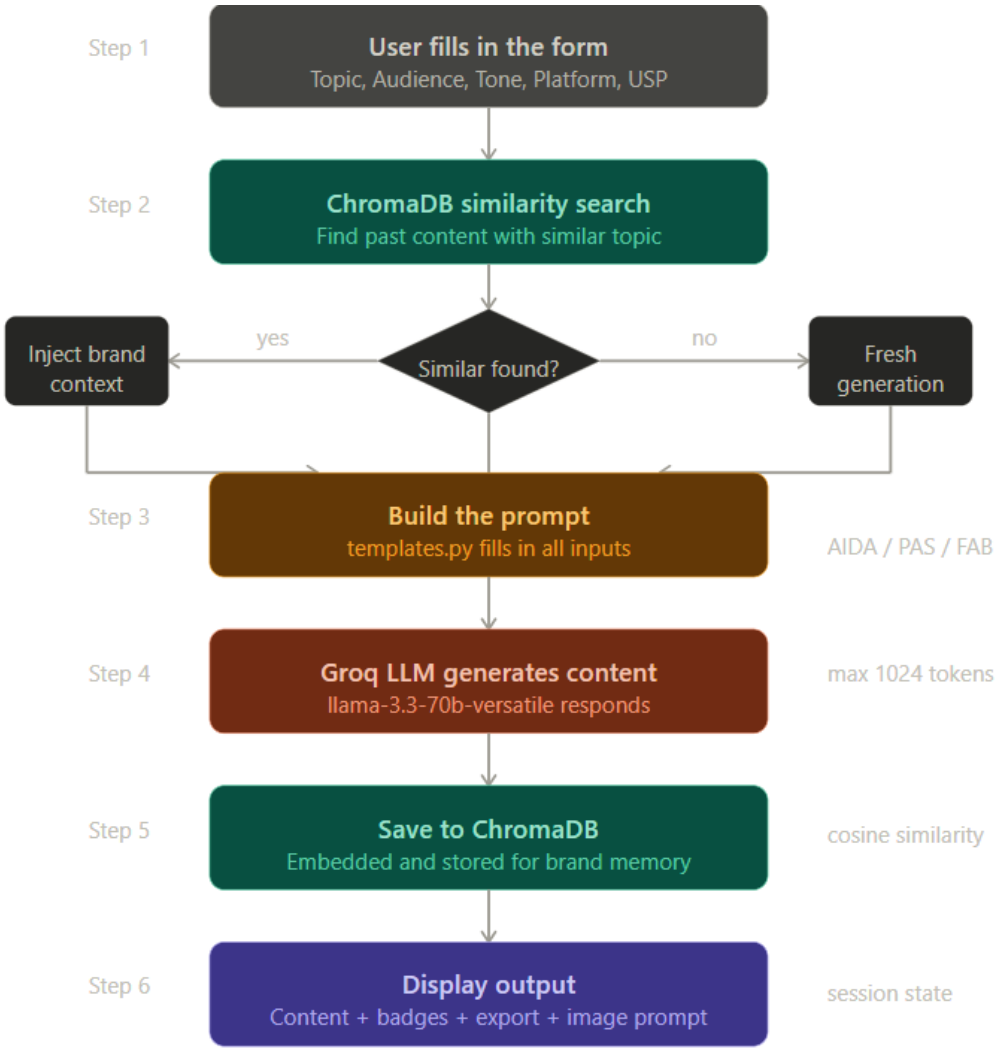


Fig.1- Process Flow

2.3 Information Flow

The system uses Session State keys (generated_content, generation_meta, image_prompt) to manage the flow of information between the UI and the processing modules. Metadata including model name and token count is exposed to the user for transparency.

2.4 Components Design

1. LLM Engine Class

- Responsibility: Sole interface with Groq API.
- Methods:
  - generate_content(): Returns a structured dict containing content and usage stats.
  - generate_image_prompt(): Creates short, high-creativity visual prompts.

2 . Content Vector Store Class

- Database: ChromaDB with all-MiniLM-L6-v2 embeddings.
- Metric: Cosine similarity is used to find relevant past content.
- Collections: Four separate collections (Ad Copy, Social Media, Email, Product Description) prevent cross-type data contamination.

2.5 Key Design Considerations

- **Caching**: Uses @st.cache_resource to instantiate services once, avoiding redundant API client or DB initializations.
- **Error Handling**: All API calls are wrapped in try/except blocks to isolate faults.

- **Similarity Threshold**: A 30% similarity filter is applied to prevent irrelevant context injection.
- **Statelessness**: Logic modules remain stateless; data persists only in the database or the UI session.

2.6 . API Catalogue

API / SDK	Endpoint / Method	Purpose
Groq SDK	chat.completions.create	Text generation and image prompting
ChromaDB	collection.query	Semantic search for similar past content
ChromaDB	collection.add	Storing documents with automated embeddings
Python-Docx	doc.save(buf)	In-memory generation of Word documents

3. References

- **Groq SDK**: For high-speed LLM text generation.
- **ChromaDB**: For local vector storage and semantic retrieval.
- **Streamlit**: For the web-based user interface.
- **Python-Docx**: For generating formatted Word documents.
